Ghulam Ishaq Khan Institute (GIKI)

Assignment# 1

Subject: Object Oriented Programming	**Course Code:** CS - 112 – Spring - 2025
Class: BS CY **Batch:** Fall – 2024	**Submission Date:** 20th February, 2025
Course Instructor: Muhammad Ahmad Nawaz	**Total Marks:** 10
Course TA:	
Name:	Registration Number:

Note (Read notes & instructions first)

- First, read the instructions and statements of each question carefully.
- Attempt all questions
- Write separate C++ programs for each problem.
- Include detailed comments explaining the logic and technical aspects of your code.
- Test your programs with various inputs to ensure correctness.
- Submit all your code files along with a README file that explains how to compile and run each program
- Upload file with Registraion#_Name
-
  - **CHEATING/ CASE** will be graded as **STRAIGHT ZERO MARKS**.

Objective:

This assignment is designed to deepen your understanding of C++ programming concepts, including **recursion**, **bit manipulation**, **enums**, **unions**, **structures**, **preprocessing**, **pointers**, and **strings**. You will implement programs that demonstrate these concepts while adhering to best practices in coding and problem-solving.

Part 1: Recursion

Conceptual Overview:

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller subproblems. It is particularly useful for problems that can be divided into similar smaller tasks, such as calculating factorials, Fibonacci sequences, or traversing tree structures.

Problem 1: Factorial Using Recursion

Write a recursive function in C++ to calculate the factorial of a number. The function should take an integer as input and return its factorial.

Technical Requirements:

- Handle edge cases (e.g., factorial of 0 or negative numbers).
- Ensure the function terminates correctly.

Example:

```
Input: 5
Output: 120
```

Problem 2: Fibonacci Sequence Using Recursion

Write a recursive function to find the nth Fibonacci number. The function should take an integer n as input and return the nth Fibonacci number.

Technical Requirements:

- Optimize the function to avoid redundant calculations (e.g., using memoization).
- Handle edge cases (e.g., n = 0 or n = 1).

Example:

```
Input: 6
Output: 8
```

Part 2: Bit Manipulation

Conceptual Overview:

Bit manipulation involves directly manipulating bits (binary digits) to perform operations efficiently. It is widely used in low-level programming, such as embedded systems, cryptography, and performance optimization.

Problem 3: Check if a Number is a Power of 2

Write a C++ program to check if a given number is a power of 2 using bit manipulation.

Technical Requirements:

- Use bitwise operations (e.g., &, |, ~, ^) to solve the problem.
- Explain the logic behind your solution.

Example:

```
Input: 16
Output: Yes
Input: 15
Output: No
```

Problem 4: Swap Two Numbers Using Bitwise XOR

Write a C++ program to swap two numbers without using a temporary variable. Use bitwise XOR to perform the swap.

Technical Requirements:

- Demonstrate how the XOR operation works to swap values.
- Handle edge cases (e.g., swapping the same variable).

Example:

```
Input: a = 5, b = 9
Output: a = 9, b = 5
```

Part 3: Enums

Conceptual Overview:

Enums (enumerations) are user-defined types that consist of a set of named integer constants. They improve code readability and maintainability by providing meaningful names to integral values.

Problem 5: Days of the Week Using Enums

Define an enum `Day` that represents the days of the week (Monday to Sunday). Write a C++ program that takes an integer input (1-7) and prints the corresponding day using the `Day` enum.

Technical Requirements:

- Use a switch-case statement to map integers to enum values.
- Handle invalid inputs gracefully.

Example:

```
Input: 3
Output: Wednesday
```

Part 4: Unions

Conceptual Overview:

A union is a user-defined data type that allows storing different data types in the same memory location. Only one member of the union can hold a value at any given time, making it memory-efficient.

Problem 6: Union to Store Different Data Types

Define a union `Data` that can hold an integer, a float, or a character. Write a C++ program to demonstrate the use of this union by assigning values to each type and printing them.

Technical Requirements:

- Explain how memory is shared among union members.
- Demonstrate the size of the union using `sizeof`.

Example:

```
Input: Assign integer value 10, float value 3.14, and character 'A'
Output:
Integer: 10
Float: 3.14
Character: A
```

Part 5: Structures

Conceptual Overview:

Structures are user-defined data types that group related variables under a single name. They are useful for representing real-world entities, such as students, employees, or geometric shapes.

Problem 7: Student Record Using Structures

Define a structure `Student` that contains the following fields:

- `name` (string)
- `roll_no` (int)
- `marks` (float)

Write a C++ program to create an array of `Student` structures, input data for 3 students, and display the details of the student with the highest marks.

Technical Requirements:

- Use a loop to input data for multiple students.
- Implement a function to find the student with the highest marks.

Example:

```
Input:
Student 1: Name = "Alice", Roll No = 1, Marks = 85.5
Student 2: Name = "Bob", Roll No = 2, Marks = 90.0
Student 3: Name = "Charlie", Roll No = 3, Marks = 78.5

Output:
Student with highest marks: Bob, Roll No: 2, Marks: 90.0
```

Part 6: Preprocessing

Conceptual Overview:

Preprocessing is a phase in C++ compilation where the preprocessor directives (e.g., `#define`, `#ifdef`, `#include`) are processed before the actual compilation. It is used for macro definitions, conditional compilation, and file inclusion.

Problem 8: Macro to Compute Square

Write a C++ program that uses the `#define` directive to create a macro `SQUARE(x)` which returns the square of `x`. Use this macro in your program to compute the square of a number.

Technical Requirements:

- Explain the difference between macros and functions.
- Demonstrate the use of the macro in your program.

Example:

```
Input: 5
Output: 25
```

Problem 9: Conditional Compilation Using Preprocessor Directives

Write a C++ program that uses the `#ifdef`, `#else`, and `#endif` directives to conditionally compile code. Define a macro `DEBUG` and use it to print debug messages if `DEBUG` is defined.

Technical Requirements:

- Show how conditional compilation can be used to include or exclude code based on macro definitions.
- Demonstrate the output with and without the `DEBUG` macro.

Example:

```
If DEBUG is defined:
Output: Debug mode is on.
Else:
Output: Debug mode is off.
```

Part 7: Pointers

Conceptual Overview:

Pointers are variables that store memory addresses. They are powerful tools for dynamic memory allocation, array manipulation, and function arguments.

Problem 10: Modify Variable Using Pointers

Write a C++ program to demonstrate the use of pointers. Create an integer variable, assign it a value, and then use a pointer to change the value of the variable.

Technical Requirements:

- Explain the relationship between pointers and memory addresses.
- Demonstrate pointer arithmetic (if applicable).

Example:

```
Input: int x = 10;
Output: Value of x after modification: 20
```

Problem 11: Swap Two Numbers Using Pointers

Write a C++ program to swap two numbers using pointers.

Technical Requirements:

- Use pointer dereferencing to swap values.
- Explain the difference between pass-by-value and pass-by-reference.

Example:

```
Input: a = 5, b = 9
Output: a = 9, b = 5
```

Part 8: Strings

Conceptual Overview:

Strings in C++ are sequences of characters stored in contiguous memory locations. They can be manipulated using pointers, arrays, or the `std::string` class.

Problem 12: Concatenate Strings Using Pointers

Write a C++ program to concatenate two strings without using the `strcat` function. Use pointers to perform the concatenation.

Technical Requirements:

- Handle memory allocation for the concatenated string.
- Ensure the resulting string is null-terminated.

Example:

```
Input: str1 = "Hello", str2 = "World"
Output: HelloWorld
```

Problem 13: Reverse a String Using Pointers

Write a C++ program to reverse a string using pointers.

Technical Requirements:

- Use pointer arithmetic to reverse the string in place.
- Handle edge cases (e.g., empty string).

Example:

```
Input: "Hello"
Output: "olleH"
```